

Multi-Agent Workflows and Team Adoption

Coordinating parallel work, and keeping an audit trail across teams and toolchains.

What You'll Learn

1

Coordinate multiple agents or subagents across a task graph

2

Decide when to parallelize versus sequence work

3

Apply repository conventions and governance across teams and toolchains

Parallelize or Sequence?

Some work parallelizes cleanly: independent services, independent tasks. Some doesn't: shared state, sequential dependencies, so the decision to parallelize has to be made per task, not assumed by default.

Delegating a focused sub-task to a second agent instance works best when the boundary is clear and the output is easy to review on its own, otherwise the coordination overhead can outweigh the time saved.

Conventions and Governance

A specs/ subdirectory per service, with specs linked to issues and pull requests, so anyone can trace a change back to the requirement that justified it.

Reviewing agent actions before execution, managing secrets and permissions, and keeping an audit trail that holds up across teams using different toolchains, so governance doesn't quietly break down the moment two teams use different tools.

Uncoordinated parallel agents create their own kind of chaos.

Two agents editing overlapping code without visibility into each other produce conflicts no single review catches early, because neither review was looking for a conflict with work happening somewhere else.

An audit trail only works if every agent's actions were reviewable in the first place, so visibility has to be designed in before you parallelize, not added afterward.

APPLYING THE CONCEPTS

GitHub Spec Kit

Workflows: Explicit Orchestration

`specify workflow run`

YAML-defined workflows chain commands, prompts, and shell steps into repeatable sequences, with gate steps for human checkpoints, so a multi-step process can be run the same way every time instead of reassembled manually.

Parallel primitive: fan-out and fan-in step types dispatch a step across a list of items and aggregate the results: a direct primitive for coordinating parallel agent work across a task graph, so parallel work doesn't have to be hand-rolled each time.

Audit trail: Every run persists state (state.json, log.jsonl) and can pause and resume from the exact point of interruption, doubling as an audit trail, so a long-running workflow doesn't have to restart from scratch after an interruption.

APPLYING THE CONCEPTS

OpenSpec

Bulk-Archive, Not Orchestration

```
/opsx:bulk-archive
```

Coordinates multiple parallel change streams at once, detecting and resolving spec conflicts between them by checking what's actually implemented, which is the closest OpenSpec gets to explicit multi-agent orchestration.

Worth knowing: OpenSpec's beta "Stores" feature extends planning across multiple repos or teams, for organizations working past a single-repo scope, worth knowing about even though it's still evolving.

Important distinction: Genuine agent-to-agent delegation is a capability of the underlying coding agent (Claude Code subagents, Copilot Fleet mode), not a feature either SDD framework owns directly, so this is a distinction worth keeping straight when evaluating either tool.

Module 7 at a Glance

	GitHub Spec Kit	OpenSpec
Orchestration primitives	fan-out / fan-in, gates, resumable state	None (conflict-aware bulk archiving only)
Multi-stream support	Workflow YAML with conditional logic	/opsx:bulk-archive
Agent delegation	Not framework-owned (agent capability)	Not framework-owned (agent capability)

Key Takeaways

- Parallelism is a property of the work, not a default setting, so it has to be decided case by case, not assumed.
- Spec Kit's workflow engine gives you explicit orchestration primitives; OpenSpec's support is lighter-weight and less structured.
- Delegation to a second agent is your coding agent's job, not the SDD framework's, so don't expect either framework to manage that hand-off for you.